Programmation Mobile

Rapport du *Projet Application iOS*

41LIDA07
Professeur : Serge Miguet

Eoin Brereton Hurley, Iliana Benchikh

March 27, 2026

1 Introduction

Dans le cadre du cours de Programmation Mobile, nous avons développé une application iOS nommée *BonSang*. Cette application permet à un utilisateur diabétique de suivre sa glycémie et d'enregistrer ses repas afin d'en analyser l'impact. Le projet a été réalisé en binôme en utilisant Swift et Xcode, avec une attention particulière portée à l'utilisation de Storyboard pour la conception de l'interface.

> L'intégralité du code est disponible dans le dépôt *GitHub* suivant : https://github.com/EoinBH/Programmation-Mobile

2 Compilation et exécution du projet

Le projet repose sur une architecture client-serveur nécessitant le lancement préalable d'un backend ainsi que la configuration de plusieurs outils externes.

2.1 Lancement du serveur backend

Le backend est implémenté en Node.js et permet d'interagir avec l'API Dexcom.

1. Se rendre dans le répertoire `dexcom-backend`
2. S'assurer que le fichier caché `.env` existe bien et contient les identifiants nécessaires *(Voir annexes pour obtenir ce fichier !)*
3. Lancer le serveur avec la commande :

```
node server.js
```

Le serveur est alors accessible en local sur le port `3000`.

2.2 Configuration de l'authentification Dexcom (OAuth)

L'authentification à l'API Dexcom repose sur le protocole OAuth 2.0.

Les identifiants nécessaires (*client ID* et *client secret*) sont stockés dans un fichier `.env` non inclus dans le dépôt GitHub pour des raisons de sécurité.

> Pour des raisons de sécurité, les clés API ne doivent jamais être publiées dans un dépôt public.

2.3 Utilisation de ngrok

L'API Dexcom impose l'utilisation d'une URL sécurisée en HTTPS. Or, un serveur local (`http://localhost`) ne satisfait pas cette contrainte. Pour contourner ce problème, nous utilisons l'outil `ngrok`, qui permet d'exposer un serveur local via une URL HTTPS publique.

> `nrgok` agit comme un tunnel sécurisé entre Internet et le serveur local, permettant de rediriger les requêtes HTTPS vers `localhost`.

2.3.1 Installation et configuration

1. Installer ngrok :

```
brew install ngrok
```

2. Ajouter le token d'authentification :

```
ngrok config add-authtoken [Voir Annexes pour Token !]
```

3. Lancer le tunnel vers le serveur local :

```
ngrok http 3000
```

Une URL HTTPS publique est alors générée et redirige vers le serveur local. (`https://india-unfightable-overbearingly.ngrok-free.dev/`)

2.4 Lancement de l'application iOS

Une fois le serveur et `ngrok` lancés, l'application peut être utilisée :

1. Lancer l'application via Xcode
2. Cliquer sur le bouton *Se connecter à Dexcom*
3. Si un avertissement ngrok apparaît, cliquer sur *Visit Site*
4. Sur la page Dexcom :
   - sélectionner un appareil (G7, One+, etc.)
   - saisir un nom d'utilisateur
   - cliquer sur *Authorize*
5. Fermer la fenêtre de navigation

3 Objectifs

Le projet s'inspire en partie de l'application *Dexcom G7*, notamment dans la manière de présenter les données de glycémie, afin de proposer une expérience utilisateur cohérente et réaliste.

3.1 Objectifs fonctionnels

L'application permet :

- de visualiser les données de glycémie,
- d'afficher ces données sous forme de graphique,
- d'enregistrer des repas avec des informations associées,
- de consulter un historique des repas.

4 Description générale

L'application comporte trois écrans principaux :

4.1 Écran principal

- Authentification via Dexcom
- Affichage des données de glycémie
- Graphique interactif

Figure 1: Écran principal

Un composant personnalisé `GraphView` a été développé. Il permet :

- un affichage dynamique des données,
- une interaction tactile,
- la visualisation de zones de risque.

4.2 Écran d'ajout d'événements (saisie manuelle)

Cet écran permet à l'utilisateur d'ajouter manuellement des événements (repas ou activités physiques), ainsi que de consulter l'historique de ces événements.

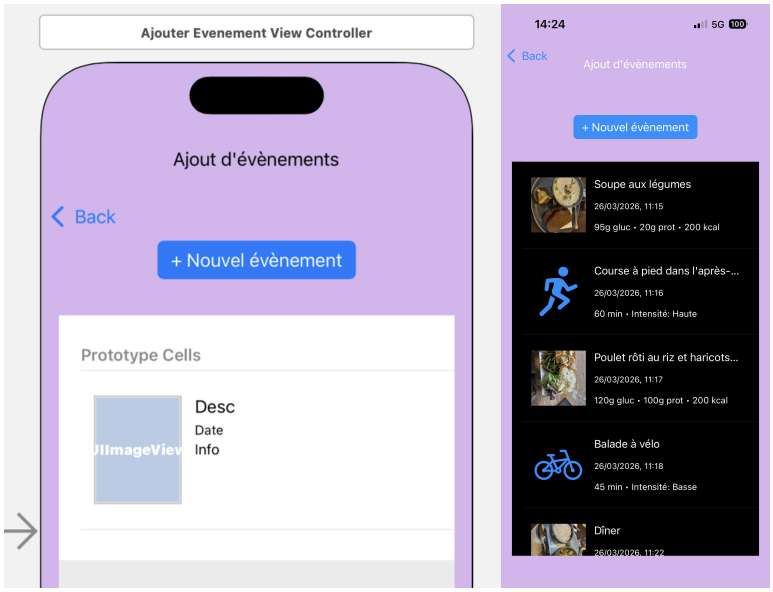

Figure 2: Écran d'ajout et de gestion des événements

4.2.1 Conception avec Storyboard

L'interface de cet écran a été entièrement réalisée à l'aide de *Storyboard*, ce qui constituait un objectif central du projet.

Les éléments suivants ont été utilisés et configurés graphiquement dans Xcode :

- une **UITableView** pour afficher la liste des événements,
- un **UIButton** permettant d'ajouter un nouvel événement,
- des **UILabel** pour afficher les informations textuelles (date, description),

- un **UIImageView** pour afficher soit une photo de repas, soit une icône d'activité.

Les contraintes *Auto Layout* ont été définies directement dans le Storyboard afin d'assurer un affichage cohérent.

Une cellule personnalisée (`MealCell`) a également été conçue via Storyboard, avec plusieurs sous-composants (image, labels), puis associée au code Swift via des *IBOutlet*. Cette approche permet de bien séparer la structure visuelle (Storyboard) et la logique applicative.

4.2.2 Cellule personnalisée

Chaque événement est affiché dans une cellule personnalisée contenant une image (photo du repas ou icône système), une description, une date formatée et des informations nutritionnelles ou d'activité.

L'affichage est dynamique : pour un repas, une image réelle est affichée et pour une activité, une icône système (*SF Symbols*) est utilisée avec une couleur adaptée.

4.2.3 Fonctionnalités

L'ajout d'un événement se fait via un bouton qui déclenche une interface de type *ActionSheet*, permettant à l'utilisateur de choisir entre un repas et une activité physique. L'utilisateur peut saisir les glucides, protéines, calories et une petite description du repas. De plus, il peut saisir le type d'activité, la durée, l'intensité et une description. Une icône est automatiquement attribuée en fonction du type d'activité (course, vélo, etc.), ce qui améliore la lisibilité de l'interface.

L'utilisateur peut supprimer un événement directement depuis la liste grâce à un geste de *swipe*. Une confirmation est demandée avant suppression, afin d'éviter toute erreur.

4.2.4 Persistance des données

Les événements sont sauvegardés localement sur l'appareil de l'utilisateur à l'aide de **UserDefaults**.

Les données sont encodées en JSON grâce au protocole `Codable`, ce qui permet :

- une sauvegarde simple,
- une restauration automatique au lancement de l'application,

- une gestion structurée des différents types d'événements.

Cette solution est adaptée à un prototype, bien qu'une solution plus robuste (Core Data) puisse être envisagée.

4.3 Gestion des repas : ajout, analyse et historisation

Cette partie de l'application permet à l'utilisateur de rechercher des aliments, composer un repas, obtenir une analyse nutritionnelle complète, puis consulter l'historique de ses repas enregistrés.

Trois écrans principaux ont été réalisés dans Xcode à l'aide de *Storyboard* :

- l'écran d'**ajout d'un repas**,
- l'écran de **résumé nutritionnel**,
- l'écran d'**historique des repas**.

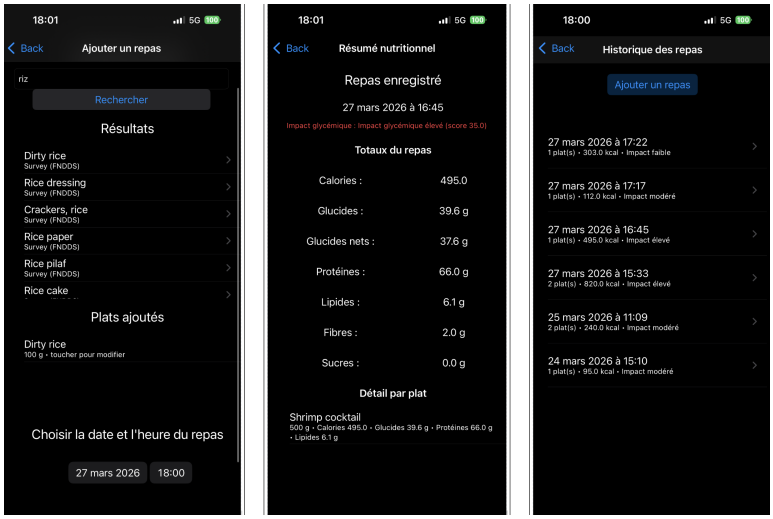

Figure 3: Ajout, analyse et historisation de repas

4.3.1 API de traduction et API nutritionnelle

Afin de permettre à l'utilisateur de rechercher un aliment en français, le backend utilise d'abord une **API de traduction**. Le texte saisi est envoyé à l'API **MyMemory Translation API**, qui traduit automatiquement la requête vers l'anglais.

Une fois la requête traduite, le backend interroge l'**API USDA FoodData Central**. Cette API nutritionnelle est utilisée à deux niveaux :

- pour **rechercher des aliments** correspondant à la requête utilisateur
- pour **récupérer les détails nutritionnels** d'un aliment précis à partir de son identifiant `fdcId`.

Les nutriments récupérés sont ensuite recalculés en fonction du grammage saisi par l'utilisateur. Le backend produit enfin :

- les valeurs nutritionnelles de chaque plat,
- les totaux du repas,
- une estimation du **score** / de l'**impact glycémique**.

4.3.2 Écran d'ajout d'un repas

Cet écran permet à l'utilisateur de construire son repas aliment par aliment.

Fonctionnement L'utilisateur saisit le nom d'un aliment, puis lance la recherche. Le frontend envoie la requête au backend, qui traduit la requête en anglais ; interroge l'API USDA et renvoie une liste de correspondances.

L'utilisateur sélectionne ensuite le bon aliment, renseigne son grammage, puis l'ajoute au repas. Il peut répéter l'opération autant de fois que nécessaire. Une fois le repas composé, il choisit la date et l'heure, puis valide. Le frontend appelle alors l'endpoint d'analyse, qui retourne le détail nutritionnel complet du repas.

4.3.3 Écran de résumé nutritionnel

Cet écran affiche le résultat de l'analyse du repas.

Fonctionnement Après l'analyse, le backend retourne les informations globales du repas ; la liste détaillée des plats analysés ; les totaux nutritionnels et une estimation de l'impact glycémique.

Ces données sont affichées de façon structurée dans l'écran de résumé. Le détail par plat permet de visualiser la contribution nutritionnelle de chaque aliment composant le repas.

4.3.4 Écran d'historique des repas

Cet écran permet de consulter l'ensemble des repas déjà enregistrés.

Fonctionnement Au chargement de l'écran, l'application interroge le backend via `/nutrition/history`. Chaque cellule affiche un résumé du repas, y compris la date, le nombre de plats, les calories et le niveau d'impact glycémique.

L'utilisateur peut ensuite ouvrir le détail nutritionnel d'un repas, modifier un repas existant ou supprimer un repas via un geste de *swipe*.

4.3.5 Persistance des données

L'historique n'est pas stocké localement dans l'application, mais côté backend dans un fichier JSON. Cette approche permet de centraliser les repas enregistrés.

4.3.6 Implémentation dans Xcode

Dans Xcode, les trois écrans ont été réalisés en *Storyboard*, puis reliés au code Swift via des *IBOutlet* et *IBAction*. Les contrôleurs principaux sont :

- `AjouterRepasViewController`,
- `MealSummaryViewController`,
- `MealHistoryViewController`.

La communication avec le backend a été centralisée dans un service Swift dédié, ce qui permet d'appeler les endpoints réseau de manière propre, de décoder les réponses JSON, de mettre à jour l'interface de façon asynchrone avec `async/await`.

5 Répartition des tâches

Le projet a été réalisé en binôme, avec une répartition des tâches permettant à chacun de travailler sur des aspects complémentaires de l'application.

5.1 Travail de Eoin

Eoin s'est principalement chargé des aspects liés à l'intégration de l'API Dexcom ainsi que de la gestion des données de glycémie. Cela inclut :

- la mise en place du flux d'authentification OAuth 2.0,
- le développement du serveur intermédiaire en Node.js,
- la récupération et le traitement des données de glycémie,
- la conception de l'écran permettant l'ajout manuel d'événements.

Son expérience personnelle avec le diabète a également permis d'orienter certaines décisions de conception, notamment en s'inspirant de l'application *Dexcom G7* afin de proposer une interface pertinente et réaliste.

5.2 Travail de Iliana

Iliana s'est concentrée sur les aspects liés aux données nutritionnelles et à l'interface utilisateur :

- intégration d'une API nutritionnelle pour enrichir les informations des repas,
- la conception des écrans permettant l'ajout et l'analyse des plats automatiquement,
- amélioration de l'affichage des données dans l'application,
- l'optimisation du graphique de glycémie (lisibilité, interaction).

Certaines parties ont été réalisées conjointement, notamment la conception globale de l'interface via Storyboard, les phases de test / débogage et l'intégration finale des différentes fonctionnalités.

6 Utilisation de l'Intelligence Artificielle

Dans le cadre de ce projet, des outils d'intelligence artificielle, notamment *ChatGPT*, ont été utilisés comme support au développement.

L'IA a d'abord été utilisée pour explorer des solutions concernant la conception du graphique de glycémie. Les propositions obtenues ont servi de base de réflexion, puis ont été adaptées et intégrées progressivement après compréhension. Elle a également été utilisée pour faciliter l'implémentation du protocole OAuth 2.0, en aidant à comprendre son fonctionnement et à structurer les échanges entre l'application et le serveur. Enfin, l'IA a servi d'outil d'assistance ponctuel pour le débogage et la résolution de problèmes.

Toutes les solutions proposées ont été testées et comprises avant intégration, l'IA ayant été utilisée comme un outil d'aide et non comme un substitut à l'apprentissage.

7 Problèmes rencontrés

Le développement du projet a soulevé plusieurs difficultés techniques. Le travail en binôme sur le Storyboard a entraîné des conflits lors du partage des fichiers, avec parfois la perte de connexions entre l'interface et le code

(`IBOutlet`, *actions*), nécessitant des corrections manuelles. L'utilisation d'*Auto Layout* a également demandé de nombreux ajustements afin d'obtenir un affichage cohérent et sans conflits de contraintes.

Le développement du graphique a été complexe, notamment pour représenter correctement les données et gérer les interactions tactiles. Enfin, la mise en place du serveur Node.js et de l'authentification OAuth a posé des problèmes de communication et de gestion des tokens, nécessitant un travail de débogage approfondi.

8 Perspectives d'amélioration

Plusieurs améliorations auraient pu être implémentées avec davantage de temps :

- **Notifications intelligentes** : alerter l'utilisateur en cas de glycémie anormale.
- **Analyse avancée des repas** : corrélation automatique entre les repas et l'évolution de la glycémie.
- **Amélioration du graphique** : ajout de zoom, affichage des évènements directement sur le graphique et meilleure personnalisation.
- **Interface utilisateur** : amélioration du design et ajout d'une icône personnalisée.
- **Compatibilité étendue** : adaptation complète à différents formats d'écran (iPad, autres iPhone).

Ces pistes permettraient de transformer ce prototype en une application plus complète et proche d'un produit réel.

9 Conclusion

Ce projet nous a permis de mettre en pratique les concepts du développement iOS, notamment la conception d'interface avec Storyboard et l'intégration d'une API externe. Il constitue une première approche concrète du développement d'applications mobiles complètes.

10 Annexes

Informations importantes par rapport au lancement du code :
Auth-token de `ngrok` :

`3AJ2xk5edIwp9KpL48pmiBbfqdk_3Mpx7xqzTxg7GKcGzQBXo`

Fichier `.env` :

```
PORT=3000

DEXCOM_CLIENT_ID=DJ8sbtcImmOLyA7w4cMujbKdTEEryYBe
DEXCOM_CLIENT_SECRET=n3tiLBTxNwMfeO3a
DEXCOM_REDIRECT_URI=https://india-unfightable-overbearingly.ngrok-free.dev/oauth/callback

USDA_API_KEY=QRG7obQgZoTXcLMpPjLGeWky1iMTuaw8okATppmT

MYMEMORY_EMAIL=benchikh.iliana@gmail.com
```